

R2.01 : Développement orienté objet

Programmation C++ & UML

BUT Informatique 2^e semestre

Eric REMY
`eric.remy@univ-amu.fr`

IUT d'Aix-Marseille, site d'Arles

Version du 28 mars 2026



Jusqu'ici, vous avez fait de la *programmation structurée* :

- On se concentre principalement sur le traitement (algorithmes, fonctions, procédures)
- On cherche à structurer les données (tableaux, structures, pile, liste, etc.)
- On organise le développement en modules (différents fichiers *.cpp et leurs *.hpp associés)
- Technique principale de 1980 à 2000.
- Correspond aux langages de l'époque : C, Basic, Pascal, FORTRAN 90, etc.

Maintenant, on va faire de la *Programmation Orientée Objet* :

- On se concentre sur les objets liés au *métier* (du client) : facture, objet 3D, personnage, salarié, voiture, etc.
- On regroupe dans chaque objet :
 - les données qui lui sont liées (comme dans une structure) ;
 - les traitements qui sont spécifiques à ces données et qui forment l'interface de l'objet.
- Chaque objet se comporte donc comme une boîte noire robuste ne présentant qu'une interface simple.
- Le but est de faire des composants solides, autonomes, faciles à maintenir, etc.
- Souvent le programmeur qui conçoit et réalise une classe n'est pas le même programmeur que celui qui va l'utiliser : le but du premier est d'empêcher le deuxième de faire des bêtises avec son boulot !
- En pratique, c'est très simple : on passe des `struct` à des `class` !

Unified Modeling Language (UML)

- Notation commune (« *unified* ») pour représenter de manière synthétique les concepts de la programmation orientée objet qui servent à la conception (« *modeling* ») des programmes.
- Pas spécifique à un langage (C++) / C'est un langage (« *language* ») en soi !
- Version 2.5.1 (2017)
- Site officiel : <https://www.uml.org/>
- En R2.01, on verra certains diagrammes (diagramme de classe, cas d'utilisation, diagramme de sequence)... mais pas tous.
- Des indications dans les diapositives du cours :
un concept C++  $\longleftrightarrow$ un concept UML 



- Définir une `class` définit un nouveau *type* qui s'ajoute à ceux connus du compilateur.
- Une **instance** (autrement dit, en français, un exemplaire) d'une `class` est une variable de ce type.
- Une `class` comporte des **données membres**  (comme les champs d'une `struct`) (aussi appelées **attributs** ) mais aussi des **fonctions membres**  (aussi appelées **méthodes** .
- Des fonctions membres spéciales servent à initialiser une instance : les **constructeurs**.
- Une fonction membre particulière est déclenchée automatiquement à la fin de vie de chaque instance : le **destructeur**.
- Grâce à la surcharge, les classes pourront apporter leurs définitions propres de certains opérateurs : `<<`, `>>`, `=`, `+`, `==`, `[ ]`, etc.
- Vous avez déjà utilisé des classes toutes faites (par exemple, `string`, `istream`, `ostream`, `fstream`, etc.)... sans trop y penser, donc les classes ne sont pas d'un usage difficile !
- ... Ce qui est plus délicat, c'est la **conception** et la **réalisation** d'une *nouvelle* classe...

Exemple d'utilisation d'une classe toute faite : la `class string`

```
1 #include <string>           // Apporte la déclaration (déjà faite) de la classe string
2 ...
3 {
4     string str1("Bonjour "); // Création d'une instance str1 de la classe string grâce au constructeur à un paramètre de type const char *.
5     string str2;             // Création d'une instance str2 de la classe string grâce au constructeur sans paramètre.
6     str2 = "monde !";        // Déclenchement de l'opérateur d'affectation « = » de la classe string avec une chaîne C en paramètre.
7     cout << str1.length();   // Affichage de la longueur de str1 grâce au déclenchement de la méthode string : :length() sur l'instance str1.
8     str1.append(str2);        // Ajoute le contenu de str2 à la suite du contenu de str1 grâce à la méthode string : :append().
9     cout << str1;            // Affiche « Bonjour monde! » grâce à la surcharge de l'opérateur « << » ayant un string en paramètre.
10 }
```

- **Instanciation** : opération réalisée par un des constructeurs et qui fabrique une nouvelle instance de la classe.
- La partie « `string ::` » du nom de la fonction sert à dire explicitement que « `append()` » est membre de `string` (et pas une fonction « classique »).
- L'opérateur « `::` » se nomme **opérateur de résolution de portée**.
- Attention : **rien de visible** dans le code au moment des déclenchements du destructeur !
- Documentation de la classe `string` (et du reste!) :
<http://www.cplusplus.com/reference/string/string/>

On va maintenant apprendre à créer une *nouvelle* classe conçue par nos soins : la classe Ratio.

Quelques rappels de mathématiques sur les rationnels :

- L'ensemble des rationnels $\mathbb{Q}$ est l'ensemble des fractions : $\{n/d : n \in \mathbb{Z} \text{ et } d \in \mathbb{N}^*\}$
- Données membres : numérateur et dénominateur
- L'implantation se fait dans :
 - Ratio.hpp : pour la déclaration de `class` elle-même et les déclarations des fonctions membres ;
 - Ratio.cpp : pour l'implantation des fonctions membres.
- Un rationnel n/d est sous *forme réduite* si et seulement si $\text{pgcd}(n, d) = 1$.
- Deux rationnels n_1/d_1 et n_2/d_2 (réduits ou non) sont égaux si $n_1 \times d_2 = n_2 \times d_1$.

- On suppose la classe Ratio déjà créée et sa déclaration disponible dans « Ratio.hpp ».

main.cpp

```
1 #include "Ratio.hpp" // Inclusion de notre fichier de déclaration de la classe (voir plus loin).
2 ...
3 {
4     Ratio r1(-3,4); // r1=-3/4 Construction à deux paramètres : -3 et 4.
5     Ratio r2(7);    // r2=7/1 Construction à un paramètre : 1. Dénominateur mis à 1.
6     Ratio r3;       // r3=0/1 Construction sans paramètre. Numérateur mis à 0 et dénominateur mis à 1.
7
8     r1.affecter(2,3); // r1 change et devient 2/3.
9     r2.afficher(cout); // Afficher la valeur de r2 (7/1) à l'écran.
10    cout << r1.valeurReelle(); // Afficher la valeur réelle approchée de r1 : 2/3 = 0,666...
11 }
```

- Si on se débrouille bien pour la conception, l'utilisation de la classe Ratio devrait être aussi *simple* que celle de la classe `string`.

Déclaration de la `class` `Ratio` : données membres

Passons maintenant à la **conception** de notre classe...

- Deux données membres seulement : le numérateur et le dénominateur.

Ratio.hpp

```
1 class Ratio {  
2     private :  
3         int num;           // Numérateur (entier relatif)  
4         unsigned int den; // Dénominateur (entier positif).  
5     public :  
6         ...  
7 }; // Attention à ne pas oublier le « ; » !
```

- Ressemble à une `struct`
- Fichier « `.hpp` » (comme d'habitude)
- On choisit de faire porter le signe du rationnel uniquement sur le numérateur.
- Ne vous focalisez pas sur « `private :` » et « `public :` » : ce sera expliqué un peu plus loin.
- **Chaque instance** aura **son** numérateur et **son** dénominateur.

Ratio.hpp (suite)

```
1 class Ratio {  
2     ...  
3 public:  
4     Ratio(int n, unsigned int d); // Constructeur à deux paramètres int et unsigned int.  
5     Ratio(int n);                // Constructeur à un paramètre int.  
6     Ratio();                     // Constructeur par défaut (c.-à-d. sans paramètre).  
7     ~Ratio();                    // Le destructeur.  
8 }; // Attention à ne pas oublier le « ; » !
```

- Chaque constructeur porte le même nom que sa classe.
- Le constructeur sans paramètre s'appelle **constructeur par défaut**. En C++98, il est indispensable de le définir si vous voulez créer un tableau d'instances de votre classe.
- Le destructeur porte le même nom que sa classe, précédé de « ~ ».
- Constructeur et destructeur sont des fonctions membres un peu particulières :
 - **Pas de type de retour** pour les constructeurs, ni pour le destructeur !
 - **Déclenchées implicitement** à la création et à la destruction de **chaque** variable du type Ratio.

- Astuce : en indiquant des valeurs par défaut aux deux paramètres du premier constructeur, on peut faire les trois constructeurs de l'exemple précédent en même temps !

Ratio.hpp (suite)

```
1 class Ratio {  
2     ...  
3 public:  
4     Ratio(int n=0, unsigned int d=1); // Constructeur à deux paramètres int et unsigned int.  
5                                         // Si on ne les précise pas, le numérateur vaudra 0 et  
6                                         // le dénominateur 1.  
7     ~Ratio(); // Le destructeur.  
8 }; // Attention à ne pas oublier le « ; » !
```

- On évite ainsi ici d'implanter le **constructeur par défaut** (celui qui n'a aucun paramètre) et le constructeur qui a un seul `int` en paramètre.

Ratio.hpp (suite)

```
1 class Ratio {
2     ...
3 public:
4     void affecter(int n,unsigned int d); // On reçoit le nouveau numérateur et dénominateur
5                                           // en paramètres.
6     void afficher(ostream&) const;      // On affichera sur un flux de sortie quelconque
7                                           // (passé par référence).
8     double valeurReelle() const;       // Pas besoin de donnée supplémentaire pour
9                                           // effectuer la division des 2 données membres.
10 }; // Attention à ne pas oublier le « ; » !
```

- Ligne 6, il n'est **pas obligatoire** de nommer les paramètres dans la déclaration : seul leur nombre et leurs **types** comptent.
- Les `const` des lignes 6 et 8 indiquent que cette fonction ne change pas l'instance de `Ratio` sur laquelle elle est déclenchée.
- Ce `const` fait partie du type complet de la fonction et peut permettre de différencier deux surcharges (avec et sans).

- « `public` : » donne accès à un membre (donnée ou fonction) à n'importe quelle fonction (par exemple, à la fonction `main()`). Ce membre fait partie de l'**interface** de la classe : la partie visible depuis l'extérieur de la « boîte noire ».
→  Indiqué par « `+` » devant l'attribut ou la méthode.
- « `private` : » interdit l'accès à ce membre qui ne peut être utilisé **que** par une des fonctions membres de la classe. C'est l'intérieur de la « boîte noire » ; personne n'est sensé y accéder directement.
→  Indiqué par « `-` » devant l'attribut ou la méthode.
- « `protected` : » ressemble à « `private` : » mais n'a d'utilité que lorsqu'on utilise l'héritage (un mécanisme important qui sera vu plus tard).
→  Indiqué par « `#` » devant l'attribut ou la méthode.
- Il existe en Java une portée de paquetage qui n'a pas d'équivalent en C++ et qui est la portée par défaut. Elle donne l'accès à toute classe ou méthode faisant partie du même *package*.
→  Indiqué par « `~` » devant l'attribut ou la méthode.

Diagramme de classe : premier exemple

Ratio
-num: int -den: unsigned int
+<<Create>> Ratio(num:int=0,den:unsigned int=1) +<<Create>> Ratio(orig:const Ratio& +<<Destroy>> ~Ratio() +affecter(num:int,den:unsigned int) +afficher(os:ostream& +valeurRéelle(): double

-  Les constructeurs sont indiqués avec le stéréotype « Create ». Je choisis d'ajouter le stéréotype « Destroy » pour le destructeur.
-  Remarquez que pour les paramètres, l'ordre est l'inverse de ce qu'on fait en C++ et qu'il faut un caractère deux-points entre les deux.
-  Similairement, le type de retour est à droite du nom de la méthode et des paramètres, après un deux-points.
-  En C++, les paramètres marqués « in » sont passés par valeur ou par référence constante ; ceux marqués « out » ou « inout » doivent être passés par référence.
-  De manière surprenante, pas de façon d'indiquer le « const » en UML !

Passons à l'implantation des fonctions !

- Maintenant qu'on a **conçu** tous les éléments de la classe Ratio (données membres, prototypes des fonctions membres, droits d'accès, etc.)...
- ... il est grand temps de passer à la **réalisation** ! C'est-à-dire écrire les définitions de toutes les fonctions membres car sans elles, il n'y a pas à proprement parler de programme !
- Définir les fonctions se fait (comme d'habitude) dans « Ratio.cpp ».
- La seule nouveauté sera de préfixer chaque nom de fonction membre par « Ratio:: » pour indiquer que cette fonction est membre de la classe « Ratio ».
- Dans une fonction membre de Ratio, « num » ou « den » faire référence aux données membres du Ratio **courant** : celui sur lequel on a déclenché la fonction membre.
- **Attention** : suite à une maladresse de votre part, une variable du même nom qu'une donnée membre (par exemple, ici « num ») définie localement à la fonction membre ou bien un paramètre de cette fonction membre, peut cacher (US : *to shadow*) la donnée membre ! Évitez de nommer vos variables locales ou bien vos paramètres avec le même nom qu'une donnée membre !

Ratio.cpp

```
1 Ratio::Ratio(int n,unsigned int d)
2   : num(n),den(d)
3   { cerr << "Ratio::Ratio(" << num << ',' << den << ")" << endl; }
4
5 Ratio::Ratio(int n) // Uniquement nécessaire si on n'a pas mis de valeurs par défaut
6   : num(n),den(1)   // au constructeur à deux paramètres.
7   { cerr << "Ratio::Ratio(" << num << ")" << endl; }
8
9 Ratio::Ratio()       // Uniquement nécessaire si on n'a pas mis de valeurs par défaut
10  : num(0),den(1)     // au constructeur à deux paramètres.
11  { cerr << "Ratio::Ratio()" << endl; }
```

- La ligne avec « : » est **uniquement possible** sur un constructeur et s'appelle une **liste d'initialisation des membres**. Elle sert à indiquer lors de la construction des données membres à quelle(s) valeur(s) ces dernières doivent être construites.
Servez-vous en : **c'est la bonne façon d'initialiser les données membres**.
- Les affichages sur `cerr` sont là à titre exceptionnel pour vous permettre en TP de bien comprendre quand ces fonctions se déclenchent ! En général, n'en mettez **jamais** !

- Depuis la norme C++ 2011, il est légal de déléguer une construction pour éviter de répéter du code.
- **Toute répétition de code est néfaste** : au minimum perte de temps, au pire, source d'erreurs.
- La délégation consiste, depuis la liste d'initialisation d'un constructeur, à appeler un *autre* constructeur de la même classe pour construire l'instance.
- On aurait donc pu, à la diapositive précédente, pour les deux dernières surcharges, écrire :

Ratio.cpp (suite)

```
1 Ratio::Ratio(int n)
2   : Ratio(n,1) // Délégation au constructeur à deux paramètres : n / 1.
3 {}
4
5 Ratio::Ratio()
6   : Ratio(0) // Délégation au constructeur à un paramètre (qui délègue à celui à deux paramètres) : 0 / 1.
7 {}
```

Ratio.cpp (suite)

```
1 Ratio::~~Ratio()  
2 { cerr << "Ratio::~~Ratio()" << endl; }
```

- Ici aussi, l'affichage sur `cerr` est **exceptionnel** !
- **Attention** : le rôle du destructeur est de **défaire** tout ce qui a été **fait** lors de la construction (et parfois plus tard).
- En particulier, il doit **rendre** (avec l'opérateur `delete`) toute la mémoire allouée dynamiquement (avec `new`) par la classe (souvent au moment de la construction).
- Pour `Ratio`, il n'y a rien de particulier à défaire... Le destructeur pourrait ici légitimement être une fonction vide !

Ratio.cpp (suite)

```
1 void Ratio::affecter(int _num, unsigned int _den)
2 { num = _num; den = _den; } // Affecte chaque paramètre à la donnée membre correspondante.
3
4 void Ratio::afficher(ostream& os) const
5 { os << num << '/' << den; }
6
7 double Ratio::valeurReelle() const
8 { return double(num) / double(den); }
```

- La fonction `affecter()` recopie ses paramètres dans les données membres.
- Attention à ne pas confondre qui est qui : paramètre $\neq$ donnée membre !
- Souvenez-vous de l'avertissement sur les variables locales qui peuvent cacher une donnée membre...
- Si vous voulez à tout prix les appeler pareil, la solution simple et classique consiste à préfixer par « `_` » le paramètre (ce qui a été fait ici).

- Fonction publique de consultation d'un membre privé.
- Cela permet de protéger une donnée membre privée contre la modification sans pour autant interdire sa consultation.
- On appelle souvent en anglais ce genre de fonction un « *getter* » car il porte souvent un nom qui commence en anglais par « *get* » (`getName()`, `getSize()`, etc.).
- Un « *getter* » qui renvoie un booléen peut aussi porter un nom qui commence par « *est* » (ou « *is* » en anglais) suivi d'un adjectif ou d'un verbe au participe passé. On peut, par exemple, imaginer la méthode « `bool Door::isOpened() const` » qui permet de tester si une porte particulière est ouverte ou non.
- Un « *getter* » ne modifie pas l'instance sur laquelle il est appelé : il devrait être marqué `const` !
- Sur le même principe, on peut aussi faire des « *setter* » publics qui modifient la valeur d'une donnée membre privée (`setName()`, `setLength()`, etc.).
- L'implantation d'un « *setter* » permet de contrôler si la modification est acceptable ou non, éventuellement en déclenchant une erreur (ou une exception).

- Souvent on marque les accesseurs « `inline` »...
- Optimise la vitesse d'une fonction très courte en demandant au compilateur de remplacer chaque appel à cette fonction... directement par ses instructions !
- Comme il n'y a plus d'appel à proprement parler, cela oblige à placer la définition de la fonction concernée dans le « `.hpp` ».
- Le modificateur `inline` n'est pas une obligation : le compilateur peut ne pas en tenir compte.
- Ce modificateur ne se limite pas aux accesseurs : il peut être appliqué à toute fonction suffisamment courte, qu'elle soit membre, par exemple `valeurReelle()` ou non-membre.

Ratio.hpp (suite)

```
1 class Ratio {
2     ...
3     public:
4         inline      int  getNum() const           { return num; } // getters
5         inline unsigned int getDen() const         { return den; }
6         inline      void  setNum(int _num)         { num = _num; } // setters
7         inline      void  setDen(unsigned int _den)
8         { if(_den !=0) den = _den; else cerr << "Impossible : diviseur nul !" << endl; }
9     };
10    ... // Plus loin, dans un .hpp ou .cpp :
11    Ratio r1(3,4); cout << r1.getNum(); r1.setDen(7); // Exemple d'usage
```

- C'est la façon générale/courante de faire cela (Java, etc.).
- À titre d'exemple, ici on empêche de mettre un dénominateur nul. C'est justement pour éviter ce genre de problème qu'il est bien de pouvoir empêcher l'utilisateur de la classe d'aller directement modifier les données membres.

Ratio.hpp (suite)

```
1 class Ratio {
2     ...
3     public:
4         inline      int    numerateur()    const { return num; } // getters
5         inline unsigned int  denominateur() const { return den; }
6         inline      int & numerateur()      { return num; } // setters
7         inline unsigned int & denominateur() { return den; }
8     };
9     ... // Plus loin, dans un .hpp ou .cpp :
10 Ratio r1(3,4); cout << r1.numerateur(); r1.denominateur() = 7; // Exemple d'usage
```

- Cette façon n'est possible qu'en C++ grâce à la syntaxe permettant de renvoyer une référence comme résultat d'une fonction !
- Il est ainsi possible de faire une affectation dans le résultat d'une fonction... puisque c'est une référence à la donnée membre !
- N'oubliez surtout pas le `const` sur les getters sinon la signature sera la même que celle des setters et la surcharge deviendrait illégale.

- Lorsque vous passez un Ratio en paramètre d'une fonction en utilisant le passage de paramètre par valeur (comme ci-dessous), vous savez qu'une copie de la variable présente dans les parenthèses de la ligne d'appel (ici r1) est faite dans une variable locale à la fonction (ici r2).

Exemple de passage de paramètre par valeur d'un Ratio

```
1 double FaireCalcul(Ratio r2)
2 { ... } // Ce qui arrive à r2 n'a pas d'effet sur r1.
3
4 int main()
5 {
6     Ratio r1(3,4);
7     double y = FaireCalcul(r1); // Recopie de r1 dans r2 au moment de l'appel.
8     ...
9 }
```

- Vous savez que tout ce qui arrivera à la variable r2 qui est **locale à la fonction** FaireCalcul () n'affectera pas r1 car r2 est **une copie** de r1.
- Par quelle « magie » fabrique-t-on cette copie d'un type complexe comme Ratio ?...

- Parmi tous les constructeur possibles, un constructeur particulier de la classe Ratio reçoit en paramètre une référence constante à un (autre) Ratio.
- C'est le **constructeur de copie**.
- Ce constructeur est tellement nécessaire que si vous ne le faites pas, le compilateur en produira un à votre place. Malheureusement, ce constructeur généré automatiquement se contente de recopier la zone de mémoire de l'ancienne instance sur la nouvelle. Ce qui est **très souvent insuffisant** (voir diapo suivante).
- Dans ce cours, vous **devrez donc toujours l'écrire vous-même**. Cette contrainte est pour **votre bien** !

Ratio.hpp (suite)

```
1 class Ratio {  
2     ...  
3 public:  
4     Ratio(const Ratio &); // Le constructeur de copie reçoit une référence constante sur la source de la copie.  
5 };
```

- Cette fonction doit créer l'instance courante (ici de la classe Ratio) en tous points identique à celle reçue en paramètre (et qui reste constante pendant la recopie).
- Chaque année, la (mauvaise) moitié des étudiants ne savait pas qui est la source et qui est la destination de la copie... (sourir). Ça fait donc une bonne question d'interro !

Ratio.cpp (suite)

```
1 Ratio::Ratio(const Ratio& source)
2 : num(source.num), den(source.den) // Construction des données membres à l'identique de source.
3 { cerr << "Ratio::Ratio( "; afficher(cerr); cerr << ")" << endl; }
```

- L'instance construite **doit être indépendante** de l'instance d'origine de la copie : quand il y a des pointeurs en données membres, **il ne suffit jamais** de recopier la valeur du pointeur d'une instance à l'autre... sinon elles pointent la même chose qu'elles partagent (tableau, etc.) ! Là aussi, c'est spécial interro...

Suppression de fonction membre créée automatiquement

- Certaines fonctions membres sont créées automatiquement par le compilateur même si vous ne les implantez pas :
 - constructeur par défaut (celui qui n'a pas de paramètre);
 - constructeur de copie (celui qui reçoit une référence constante à une instance de la classe);
 - constructeur par déplacement (nouveau C++ 2014, ne sera pas abordé);
 - opérateur d'affectation (« operator=() » vu au prochain cours);
 - opérateur d'affectation par déplacement (nouveau C++ 2014, ne sera pas abordé);
 - destructeur.
- Puisqu'il ne suffit pas de ne pas les créer, comment faire pour les **supprimer** vraiment ?
- Il faut l'indiquer dans la déclaration de classe en utilisant le mot-clé « delete » (qui n'a ici rien à voir avec l'allocation dynamique !):

Suppression de fonctions membres

```
1 class Ratio {  
2     Ratio() = delete;           // Interdiction de construire sans paramètre.  
3     Ratio(const Ratio&) = delete; // Interdiction de construire une copie.  
4 };
```

Pointeur sur l'instance courante : `this`

- L'instance courante :
 - pendant l'appel à `Truc::exemple()` de la ligne 6, c'est `t1` ;
 - pendant l'appel à `Truc::exemple()` de la ligne 7, c'est `t2`, etc.

Exemple utilisant le mot clé `this`

```
1 Truc* Truc::exemple()  
2 { return this; }  
3 ...  
4 { // Plus loin  
5   Truc t1(...); Truc t2(...); Truc *pt;  
6   pt = t1.exemple(); // Ici équivalent compliqué à pt = &t1;  
7   pt = t2.exemple(); // Ici équivalent compliqué à pt = &t2;
```

- Besoin : nommer l'instance courante (**en totalité**, et pas seulement une donnée membre) depuis l'**intérieur** d'une fonction membre.
- Le mot clé `this` donne l'**adresse** de l'instance courante.
- Donc `*this` **est** l'instance courante durant l'appel.

- Dérogation : donner à une fonction **non-membre d'une classe Truc** l'accès aux membres privés de la classe Truc.
- Ne pas en abuser (choix de conception : peser le pour et le contre)
- Déclarer dans Truc la fonction qui doit avoir l'accès (c'est son identité) en rajoutant le mot clé « friend » devant :

Truc.hpp

```
1 class Truc {  
2     private:  
3     int i; // Accès interdit  
4     ...  
5     friend void fonction(Truc t);  
6 };
```

MesFonctions.cpp

```
1 void fonction(Truc t) // Non-membre de Truc  
2 {  
3     t.i = 10;          // On peut se servir des  
4 }                      // membres privés de Truc ici.
```

- Également possible de déclarer amie une fonction **membre d'une autre classe**.

- Généralisation à **toutes** les fonctions membres d'une (**autre**) classe Machin :

Truc.hpp

```
1 class Truc {  
2     private:  
3         int i; // Accès interdit  
4     ...  
5     friend class Machin;  
6 };
```

Machin.cpp

```
1 void Machin::fonction(Truc t)  
2 {  
3     t.i = 10;           // On peut se servir des  
4                         // membres privés de Truc ici.
```

- Ici aussi : ne pas en abuser ! C'est une bonne idée lorsque deux classes « marchent » et sont conçues ensemble (par exemple, Maillon et ListeChaine).
- Remarque : l'amitié n'est **pas réciproque** (à moins de rajouter une déclaration d'amitié symétrique en plus dans Machin) !

Surcharge d'opérateurs : introduction

- Chaque opérateur (`=`, `==`, `+`, `-`, `*`, `/`, `%`, `<<`, `>>`, `[ ]`, etc.) peut être surchargé pour **vos** classes.
- Pour un opérateur `+`, il faut par exemple réaliser une fonction `operator+()`. Et ainsi de suite...
- Deux catégories de surcharges :
 - Fonctions membres (lorsque l'instance courante joue un rôle privilégié);
 - Fonction non-membre (lorsque l'instance courante ne joue pas de rôle privilégié).
- Si vous aimez être verbeux, vous pourriez remplacer « `a = b;` » par « `a.operator=(b);` » (fonction membre) et « `b+c;` » par « `operator+(b,c);` » (fonction non-membre).
- **Vous ne devez pas improviser** le prototype de la fonction à réaliser ! Cherchez le cours (non-exhaustif) ou une documentation !

Surcharge d'opérateurs : opérateur d'affectation

- L'opérateur d'affectation `operator=()` **doit être obligatoirement réalisé** (comme le constructeur de copie) : sans lui, il serait **impossible** d'affecter une instance à une autre de la classe.
- Doit être déclaré puis réalisé en fonction membre :

Ratio.cpp

```
1 Ratio& Ratio::operator=(const Ratio & src)  
2 { num = src.num; den = src.den; return *this; }
```

- La fonction reçoit une référence constante à la source de l'affectation. La destination est l'instance courante.
- **Ce n'est pas un constructeur** (puisque la destination existe déjà avant) !
- La fonction renvoie toujours une référence sur l'instance courante (« `*this` »).
- Affecter une instance n'est donc pas nécessairement aussi facile ou rapide que ce que vous pourriez croire...
- Si utile, mettre en facteur les instructions communes au constructeur de copie et à l'`operator=` dans une fonction membre privée.

Surcharge d'opérateurs : opérateurs arithmétiques avec accumulation

- Cas des opérateurs arithmétiques combinés à une affectation ($+=$, $-=$, $*=$, $/=$, $\%=$, $<<=$, $>>=$, $\&=$, $|=$, $\wedge=$, etc.).
- Rappel : $a+=b$; $\Leftrightarrow a = a + b$;
- Similaires à l'opérateur d'affectation `operator=()`
- Doivent être déclarés puis réalisés en fonctions membres :

Ratio.cpp

```
1 Ratio& Ratio::operator*=(const Ratio & src)
2 { num *= src.num; den *= src.den; return *this; }
```

- La fonction reçoit une référence constante à la source de l'affectation. La destination est l'instance courante.
- La fonction renvoie une référence sur l'instance courante (« `*this` »).

Surcharge d'opérateurs : pré- et post-incrémentation ou décrémentation

- Rappel : $b = ++a$; $\Leftrightarrow a = a + 1$; $b = a$; et $b = a++$; $\Leftrightarrow b = a$; $a = a + 1$;
- Doivent être déclarés puis réalisés en fonctions membres :

Ratio.cpp

```
1 Ratio& Ratio::operator++()           // Pré-incrémentation
2 { num+=den; return *this; }         // Incrémentation et renvoi de l'instance courante
3
4 Ratio Ratio::operator++(int)          // Post-incrémentation
5 { Ratio res(*this);                 // Copie de l'instance courante
6   num += den;                       // Incrémentation de l'instance courante
7   return res; }                     // Renvoi de la copie
```

- Le `int` ne sert qu'à différencier les deux signatures au sens de la surcharge. La valeur du paramètre doit être ignorée.
- On aurait pu espérer que les concepteurs aient simplement choisi plutôt `++operator()` pour la première !

Surcharge d'opérateurs : accès indexé

- Supposons l'existence d'une classe `ConteneurSequentielDouble` contenant une séquence de `double`
- Les deux opérateurs suivants permettent de donner à une instance un comportement de tableau :
`ConteneurSequentielDouble cs1 (...); cout<< cs1[0]; cs1[0]=25;`
- Surcharges similaires aux **accesseurs** façons **C++**
- Doivent être déclarés puis réalisés en fonctions membres :

Ratio.cpp

```
1 // On suppose qu'un membre privé « double element[...] » contient les éléments de la séquence.
2 const double& ConteneurSequentielDouble::operator[](int i) const // Consultation
3 { return element[i]; }
4
5 double& ConteneurSequentielDouble::operator[](int i) // Modification
6 { return element[i]; }
```

- Le néologisme « foncteur » est une traduction de l'anglais « functor ». Il désigne une classe dont chaque instance peut être utilisée comme une fonction.
- Les surcharges de l'`operator()` doivent être réalisés en fonctions membres :

MonFoncteur.cpp

```
1 double MonFoncteur::operator()(double x) // Foncteur  $\mathbb{R} \rightarrow \mathbb{R}$   
2 { ... } // Le calcul peut aussi bien se servir du paramètre x que des données membres !
```

- On peut utiliser une instance comme si c'était une fonction **quelconque** :
`MonFoncteur f1(...); double y = f1(3.1415);` // Construction de f1 puis pseudo-appel de f1()
- Il est évidemment possible de donner autant de surcharges différentes que désiré : on n'est pas limité aux fonctions de `double` vers `double`.

- Le mot « itérateur » est la traduction de « iterator ».
- Surcharger l'`operator*`() (l'indirection, pas la multiplication) permet d'utiliser une instance pour toute opération réclamant un pointeur classique.
- Cette idée est très importante pour la manipulation de la bibliothèque standard de patrons (STL) et sera vue plus en détail à la fin du module.

- Pour toutes les autres surcharges qui doivent être faites en fonction non-membres, on aura le problème d'accès aux données membres...
- Il est donc **plus simple** de déclarer ces fonctions amies.
- Ce n'est cependant **pas strictement indispensable**.
- Merci donc de ne pas parler de « surcharges en fonctions amies » mais de « surcharges en fonctions non-membres »

Surcharge d'opérateurs : opérateurs d'extraction et d'injection sur des flux

- Cas des opérateurs << (**injection** sur un flux) et >> (**extraction** depuis un flux).
- Doivent être déclarés puis réalisés en fonctions non-membres :

Ratio.cpp

```
1 ostream& operator<<(ostream& os, const Ratio & src)    // non-membre
2 { os << src.num << '/' << src.den; return os; }      // Nécessite friend pour l'accès
3
4 istream& operator>>(istream& is, Ratio & dest)       // non-membre
5 { is >> dest.num ; dest.den=1;                       // Nécessite friend pour l'accès
6   char c='X'; is>>c; if(c=='/') is >> dest.den;      // !!! NON TESTÉE
7   return is; }
```

- La fonction reçoit une référence sur le flux à utiliser. Elle n'est jamais constante car, qu'on lise ou qu'on écrive, le flux est modifié !
- L'injection reçoit une référence constante sur le Ratio à écrire (il ne change pas) ; l'extraction, une référence variable (forcément le Ratio devrait changer !).
- La fonction renvoie toujours une référence sur le flux. C'est ce qui permet d'enchaîner plusieurs injection/extractions à la suite...
- Remplace la fonction `void Ratio::afficher(ostream&) const;...` en mieux !

Surcharge d'opérateurs : opérateurs arithmétiques et logiques binaires

- Cas des opérateurs arithmétiques binaires (+, -, *, /, %, etc.) mais aussi des comparateurs binaires (==, !=, <, <=, etc.).
- Doivent être déclarés puis réalisés en fonctions non-membres :

Ratio.cpp

```
1 Ratio operator*(const Ratio& a, const Ratio& b)    // non-membre
2 { Ratio res(a.num*b.num,a.den*b.den); return res; } // Nécessite friend pour l'accès
3
4 bool operator==(const Ratio& a, const Ratio& b)    // non-membre
5 { return (a.num*b.den == b.num*a.den); }           // Nécessite friend pour l'accès
```

- Les fonctions reçoivent des références constantes pour éviter les copies inutiles.
- Les types peuvent être adaptés à vos besoins.

- Jusqu'ici vous avez appris la différence entre une déclaration de fonction et une définition de fonction.
- Jusqu'ici, vous n'avez fait que des **définitions** de variables qui **créent** une variable.
- Comment peut-on alors **annoncer** l'existence d'une variable (globale) définie ailleurs ?
- Réponse : grâce au mot-clé `extern` !

Utilisation de `extern`

```
1 extern int i; // i est une variable entière définie ailleurs et
2 extern Ratio r; // r est une instance de Ratio définie ailleurs.
```

- Pour la même raison que pour les fonctions, de telles déclarations se placent en général dans un fichier « `*.h` » ou « `*.hpp` ».
- D'usage courant en C, `extern` n'a presque pas cours en C++ pour la simple raison **que les variables globales sont à éviter** !

- Quatre usages différents de `static` :
 - pour une donnée ou une fonction globale / non-membre ;
 - pour une variable locale ;
 - pour une donnée ou fonction membre.
- Ne les confondez pas : ils sont généralement **mutuellement exclusifs** !

Spécificateur `static` sur une variable ou une fonction globale

- Une variable ou une fonction globale est normalement visible depuis n'importe quel point du programme.
- ... même si il faut la déclarer pour qu'elle soit connue en dehors de son fichier de définition.
- `static` empêche que le symbole correspondant (le nom de la variable ou de la fonction) soit exporté :
 - la variable ou la fonction existe bien mais
 - elle n'est pas **visible** en dehors de son fichier de définition !
- Le seul usage que vous pourriez en avoir serait de faire plusieurs fonctions d'exactly même signature (ce qui est normalement interdit au sens de la surcharge), une par fichier « *.cpp ».

Spécificateur `static` sur une variable locale

- Évite qu'une variable locale soit détruite et recrée à chaque appel : **persistance**.
- Grosso modo, c'est une variable locale pour la visibilité et globale pour la durée de vie !
- Permet de créer une fonction qui compte les appels :

Variable locale `static`

```
1 unsigned int compteurDAppels()
2 {
3     static unsigned int compteur = 0; // important : il faut l'initialiser!
4     return ++compteur;
5 }
6 ...
7 { // Ailleurs...
8     cout << compteurDAppels() << endl; // Affiche 1
9     cout << compteurDAppels() << endl; // Affiche 2
10    cout << compteurDAppels() << endl; // Affiche 3, etc.
11 }
```

- Ne pas oublier l'initialisation puisqu'elle n'est faite qu'une seule fois !

- Habituellement, une donnée membre est un **membre d'instance** : chaque instance de la classe obtient son instance de la donnée membre. Pour une fonction membre, cela signifie qu'on ne peut l'appeler que sur une instance donnée de la classe.
- Avec `static`, il est possible au contraire de réclamer un **membre de classe** : la donnée ou la fonction n'est pas attachée à chaque instance mais **à la classe elle-même**.
- Il n'y a donc qu'une seule donnée membre de classe partagée par toutes les instances de la classe en même temps. Pour une fonction membre de classe, elle pourra être déclenchée sur la classe et pas nécessairement sur une instance.
-  En UML, les membres de classe sont soulignés.
- Exemple : compteur d'instances d'une classe

Membres `static`

```
1 class Machin {
2     private:
3         int i; // donnée membre de chaque instance de la classe Machin
4         static unsigned int compteur; // donnée membre de la classe Machin
5     public:
6         Machin(); // Constructeur par défaut.
7         static unsigned int combien(); // fonction membre de la classe Machin
8         static void reset(unsigned int value=0); // idem
9 };
```

- N'oubliez pas d'initialiser la donnée membre de classe !
- Il faut le spécificateur de portée « `Machin::` » pour désigner le bon nom.

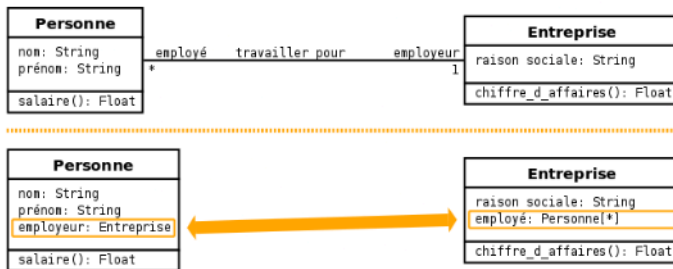
Membres `static`

```
1 unsigned int Machin::compteur=0; // donnée membre de la classe Machin
2
3 Machin::Machin()
4 : i(...) // Construit ses membres d'instance
5 { ++compteur; } // Incrémente le membre de classe.
6
7 unsigned int Machin::combien()
8 { return compteur; }
9
10 void Machin::reset(unsigned int value)
11 { compteur = value; }
12
13 int main() {
14     cout << Machin::combien() << endl; // Affiche 0
15     Machin m1,m2; // Crée deux instances
16     cout << Machin::combien() << endl; // Affiche 2
17     Machin::reset(); // Remet le compteur à zéro (par défaut)
18     cout << Machin::combien() << endl; // Affiche 0 de nouveau
19     ...
20 }
```

- Comment adapter l'exemple pour qu'il compte les instances actuellement en mémoire ?

Relations entre classes

- Dans un diagramme de classe UML, les classes peuvent être liées entre elles par des traits qui indiquent une relation particulière entre ces classes.
- Une relation peut être binaire, ternaire... n-aire.
- Elle peut porter quelques mots pour la décrire.
- Elle peut comporter une **cardinalité** c'est-à-dire préciser combien d'instances sont concernés par la relation de chaque côté. Le symbole « * » signifie n'importe quelle valeur, y-compris nulle.



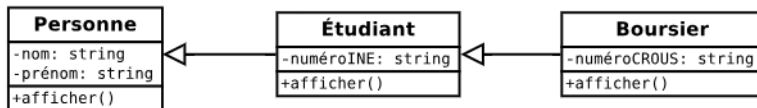
- La relation se traduit par des données membres du type de l'autre classe.

- La composition, c'est déjà ce que vous avez déjà fait : vous connaissez !
- Permet de définir une classe à partir de types existants (dont éventuellement des classes)
- Votre nouvelle classe **est composée** de ces nouveaux éléments.
- L'existence de la classe composée **est liée** à celle de ses composants. $\implies$ Les composants sont détruits en même temps que la classe composée.
- Exemple : une voiture est composée d'un moteur, de quatre (instances de) roues, etc.
-  La composition se note avec un lien entre classe comportant un $\blacklozenge$ du côté de la classe composée.
-  Il suffit de mettre des instances des classes composantes dans la classe composée.

- Permet de définir une classe à partir de types existants (dont éventuellement des classes)
- Votre nouvelle classe **est composée** de ces nouveaux éléments.
- L'existence de la classe agrégée **n'est pas liée** à celle de ses éléments. $\implies$ Les éléments ne sont pas forcément détruits en même temps que la classe agrégée. Cette dernière peut même n'en contenir aucun exemplaire.
- Exemple : un album photo contient (éventuellement) des photos... mais il est vide quand vous l'achetez et vous êtes libre d'en sortir les photos pour les ranger ailleurs.
-  L'agrégation se note avec un lien entre classe comportant un $\diamond$ du côté de la classe agrégée.
-  Pour dissocier la durée de vie, on utilisera en général un pointeur sur chaque instance des classes agrégées au sein de la classe agrégeante. Le pointeur peut donc être nullptr s'il n'y a pas d'instance agrégée.

Héritage (1/2)

- Hériter permet de récupérer toutes les propriétés (données et fonctions membres) de son ancêtre et d'en rajouter de nouvelles.
- Votre nouvelle classe **est une** variante *spécialisée* de l'ancienne.



- Un Boursier **est un** Etudiant qui **est une** Personne ! ($\neq$ « est composé d'un »)
- Chaque instance de Boursier a donc un nom, un INE et un numéro de dossier du CROUS.
- Vocabulaire :
 - Etudiant **dérive** de Personne, qui est sa classe **parente** (en anglais, **superclass**).
 - Etudiant est une **classe fille** de Personne.
-  Le lien d'héritage est indiqué par une flèche à tête triangulaire blanche $\triangleright$ qui part de la classe dérivée et va vers la classe mère.

- En C++, on va faire de l'héritage public :

Personne.hpp

```
1 class Personne {  
2     string nom, prenom;  
3     public:  
4     void afficher();  
5 };
```

Etudiant.hpp

```
1 class Etudiant  
2 : public Personne  
3 {  
4     string numeroINE;  
5     public:  
6     void afficher();  
7 };
```

Boursier.hpp

```
1 class Boursier  
2 : public Etudiant  
3 {  
4     string numeroCROUS;  
5     public:  
6     void afficher();  
7 };
```

- Possible de surcharger les fonctions et données membres pour les spécialiser, par exemple ici `afficher()`). On voit celle qui est la plus proche. Si on veut en utiliser une autre, il faut préciser laquelle avec l'opérateur de résolution de portée « `::` ». Par exemple, depuis `Boursier`, on peut appeler `Personne::afficher()` pour n'afficher ce boursier que comme une personne.

- Puisque tout Etudiant est aussi une Personne...

Classe Perso

```
1  Personne pers (...); // On crée une instance de Personne
2  Etudiant etu (...); // et une de Etudiant :
3  pers = etu; // Légal car un étudiant est aussi une personne
4              // mais tout ce qui est spécifique à étudiant n'est pas copié.
5  etu = pers; // Illégal : toute personne n'est pas un étudiant.
6
7  Personne *ppers = &etu; // Légal car l'adresse d'un étudiant est aussi d'adresse d'une personne
8  Etudiant *petu = &pers; // Illégal car l'inverse n'est pas vrai!
```

- Un « up-casting » : vous convertissez en une classe au-dessus dans l'arbre d'héritage.

- Rappel : trois classes de protection des données membres : `public :`, `private :`, `protected :`
- Dans le cas de l'héritage **public** :

Héritage public :

```
1 class A {  
2 public:  
3     int a;  
4     void fa();  
5 protected:  
6     int b;  
7 private:  
8     int c;  
9 };  
10  
11 class B : public A {  
12 public:  
13     void fb();  
14 };
```

- a et fa() sont visibles depuis n'importe où.
- c n'est visible que depuis A:: et donc ni dans B:: ni dans :: (c'est-à-dire c est utilisable dans fa() mais pas dans fb() ni main()).
- b est intermédiaire : visible depuis A:: et dans B:: mais pas dans :: (c'est-à-dire b est utilisable dans fa() et dans fb() mais pas main()).
- C'est 95% des cas d'héritage.

- Dans le cas de l'héritage privé :

Héritage private :

```
1  class A {  
2  public:  
3      int  a;  
4      void fa();  
5  protected:  
6      int  b;  
7  private:  
8      int  c;  
9  };  
10  
11  // Dans la ligne qui suit  
12  // « private » pourrait  
13  // être sous-entendu.  
14  class B : private A {  
15  public:  
16      void fb();  
17  };
```

- Seule fb() est publique dans B::.
- Les membres (données ou fonctions) publiques de A:: deviennent privés dans B::.
- Les membres protégés ou privés de A:: deviennent inaccessibles depuis B:: (il faut passer par les méthodes publiques devenues privées, point ci-dessus).

- Dans le cas de l'héritage privé :

Héritage private :

```
1 class A {  
2 public:  
3     int a;  
4     void fa();  
5 protected:  
6     int b;  
7 private:  
8     int c;  
9 };  
10  
11 // Dans la ligne qui suit  
12 // « private » pourrait  
13 // être sous-entendu.  
14 class B : private A {  
15 public:  
16     void fb();  
17 };
```

- Permet de simplifier l'interface d'une classe trop générale : on souhaite camoufler l'existence des méthodes internes.
- Développement rapide en « qui peut le plus, peut le moins ».
- Casse la propriété « est un » qui devient « est implémenté en terme de », c'est-à-dire qu'un A est implémenté grâce à un B mais **c'est un secret** !
- C'est donc **incompatible avec le polymorphisme** qui nécessiterait de pouvoir up-caster (qu'un B soit encore publiquement un A).
- C'est 4,9% des cas d'héritage (pour le 0,1%, c'est l'héritage protected...).

- Héritage multiple (exemple : un hydravion *est un* avion et *est un* bateau !)
- Héritage `protected`.
- Pointeur sur les membres de classe.

- Soient deux classes : B hérite de A :

Classe A

```
1 // A.hpp
2 class A {
3     int val1;
4 public:
5     A(int _val1=10);
6 };
7
8 // A.cpp
9 A::A(int _val1)
10 : val1(_val1)
11 {}
12
```

Classe B

```
1 // B.hpp
2 class B : public A {
3     int val2;
4 public:
5     B(int _val1, int _val2); // Deux paramètres
6 };
7
8 // B.cpp
9 B::B(int _val1, int _val2)
10 : A(_val1), // Construction de la classe mère : A.
11   val2(_val2) // Construction du membre spécialisé
12 {...}
```

- Indiquer la construction de la classe parente dans la liste d'initialisation de la classe fille.
- La construction de la classe mère a lieu **avant** le reste de la liste d'initialisation et du bloc « {...} ».

- Soient trois classes : C hérite de B, qui hérite de A :

Trois classes

```

1 class A {
2     int a;
3 public:
4     A(int _a) : a(_a) { cout<<"A::A("<<a<<""); }
5     ~A() { cout<<"A::~~A()"; }
6 };
7
8 class B : public A { // Un B est un A spécialisé
9     int b;
10 public:
11     B(int _a, int _b) : A(_a), b(_b) { cout<<"B::B("<<b<<""); }
12     ~B() { cout<<"B::~~B()"; }
13 };
14
15 class C : public B { // Un C est un B spécialisé
16     int c;
17 public:
18     C(int _a, int _b, int _c) : B(_a, _b), c(_c) { cout<<"C::C("<<c<<""); }
19     ~C() { cout<<"C::~~C()"; }
20 };
21
22 { C c(1,2,3); // Construction d'une instance c de C
23 ...
24 } // Destruction de l'instance c

```

- Constructeurs appelés dans l'ordre logique de construction
- Destructeurs appelés dans l'ordre inverse (logique !)
- Affiche donc :

```

1 A::A(1)
2 B::B(2)
3 C::C(3)
4 ...
5 C::~~C()
6 B::~~B()
7 A::~~A()

```

- Le choix d'une fonction membre est fait au moment de la compilation
- C'est donc un choix **statique** ($\neq$ dynamique, c'est-à-dire à l'exécution)
- Le choix dépend du type de l'objet sur lequel on déclenche la méthode

Choix **statique** de méthode

```
1  class A {  
2      public:  
3          void f() { cout<<"A::f()"<<endl; }  
4  };  
5  class B : public A {  
6      public:  
7          void f() { cout<<"B::f()"<<endl; }  
8  };  
9  
10 // main.cpp  
11 int main()  
12 {  
13     A a; B b;  
14     a.f();      // écrit « A::f() » (sans surprise puisque a est un A)  
15     b.f();      // écrit « B::f() » (sans surprise puisque b est un B)  
16     A *p = &a;  // p pointe sur a  
17     p->f();     // écrit « A::f() » (sans surprise puisque p pointe sur un A)  
18     p = &b;     // p pointe sur b (qui est aussi un A).  
19     p->f();     // écrit « A::f() » puisque p pointe sur un A!  
20     return 0;  
21 }
```

- Le choix est fait en fonction du seul type de p tel qu'il est déterminé **à la compilation**.
- Puisque p est un A*, c'est A::f() qui est déclenchée.

- Et si on voulait que ça affiche quand même B::f() ? C'est-à-dire que deux appels identiques donnent un résultat différent suivant le type pointé par p à l'exécution...
- Il faut du **polymorphisme** !

Choix **dynamique** de méthode

```
1 class A {  
2     public:  
3         virtual void f() { cout<<"A::f()"<<endl; }  
4 };  
5 class B : public A {  
6     public:  
7         void f() { cout<<"B::f()"<<endl; }  
8 };  
9  
10 // main.cpp  
11 int main()  
12 {  
13     A a; B b;  
14     A *p = &a; // p pointe sur a  
15     p->f();    // écrit « A::f() » (sans surprise puisque p pointe sur un A)  
16     p = &b;    // p pointe sur b  
17     p->f();    // écrit « B::f() » car p pointe un B et que f() est virtuelle  
18     p->A::f(); // écrit « A::f() » (on peut forcer le choix vers la classe mère)  
19     return 0;  
20 }
```

- Marquer la fonction f() de la classe parente avec le mot-clé « virtual ».
- Toutes les fonctions **de même signature** des classes filles sont alors virtuelles; pas besoin de remettre « virtual ».
- Chaque instance comporte un pointeur (caché) supplémentaire (vers la virtual table) ⇒ +8 octets
- Chaque appel nécessite la consultation de la vtable pour trouver « la bonne » fonction ⇒ plus lent !

- Étudiez l'exemple suivant dans lequel vous pensez avoir tout bien fait !

Trois classes

```
1 class A {  
2     int *p;  
3 public:  
4     A() { p=new int [2]; cout<<"A() "; }  
5     ~A() { delete [] p; cout<<"~A()"<<endl; }  
6 };  
7  
8 class B : public A {  
9     int *q;  
10 public:  
11     B() { q=new int [1023];  
12         cout<<"B() et q="<<q; }  
13     ~B() { delete [] q; cout<<"~B() "; }  
14 };  
15  
16 int main()  
17 {  
18     for(int i=0;i<6;i++)  
19         { A *pa= new B(); delete pa; }  
20 }
```

- Affiche :

```
1 A() B() et q=0x5821c ~A()  
2 A() B() et q=0x5921c ~A()  
3 A() B() et q=0x5a21c ~A()  
4 A() B() et q=0x5b21c ~A()  
5 A() B() et q=0x5c21c ~A()  
6 A() B() et q=0x5d21c ~A()
```

- B::~~B() n'est jamais appelé !
- Perte de 1023×4 octets à chaque tour !
- Morcellement de la mémoire
- Choix **statique** du destructeur !

- Évidemment il faut que le destructeur soit `virtual` :

Trois classes

```

1 class A {
2     int *p;
3 public:
4     A() { p=new int [2]; cout<<"A() "; }
5     virtual ~A() // Ajout de virtual
6     { delete [] p; cout<<"~A()"<<endl; }
7 };
8
9 class B : public A {
10     int *q;
11 public:
12     B() { q=new int [1023];
13         cout<<"B() et q="<<q; }
14     ~B() { delete [] q; cout<<"~B() "; }
15 };
16
17 int main()
18 {
19     for(int i=0;i<6;i++)
20     { A *pa= new B(); delete pa; }
21 }

```

- Affiche :

```

1 A() B() et q=0x5821c ~B() ~A()
2 A() B() et q=0x5821c ~B() ~A()
3 A() B() et q=0x5821c ~B() ~A()
4 A() B() et q=0x5821c ~B() ~A()
5 A() B() et q=0x5821c ~B() ~A()
6 A() B() et q=0x5821c ~B() ~A()

```

- `B::~B()` est bien appelé : choix **dynamique** du destructeur !
- Allocations toujours à la même adresse puisqu'elle est libre.
- Dès qu'on commence à utiliser du polymorphisme, 9 fois sur 10, il faut que le destructeur soit polymorphe...

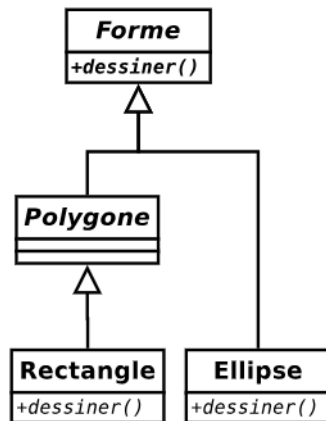
- « `override` » placé après le prototype dans la déclaration/définition de la fonction indique **explicitement** que la fonction membre de cette sous-classe redéfinit la fonction membre de sa classe parente qui **doit** être virtuelle.
- On peut également interdire la surcharge ultérieure avec « `final` ».

```
1 class A {  
2     public:  
3         virtual void f1();  
4         virtual void f3(int);  
5         void f4();  
6         virtual void f5();  
7 };  
8 class B : public A {  
9     public:  
10        void f1() override; // OK : surcharge de f1() de la classe A.  
11        void f2() override; // Erreur : n'existe pas dans A!  
12        void f3() override; // Erreur : la signature ne correspond pas!  
13        void f4() override; // Erreur : pas virtuelle dans A!  
14        void f5() final;    // OK : surcharge de f5() de la classe A.  
15 };  
16 class C : public B {  
17     public:  
18        void f1() override; // OK : surcharge de f1() de la classe B.  
19        void f5() override; // Erreur : final dans B!  
20 };
```

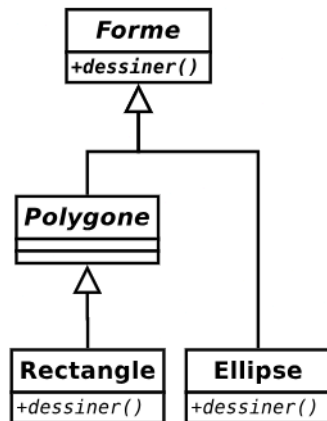
- Il est interdit d'hériter d'une classe comportant un « `final` » placé après sa déclaration.

```
1 class A final {  
2     ...  
3 };  
4  
5 class B : public A { // Erreur : la classe A ne peut pas être dérivée  
6     ...             // par héritage.  
7 };
```

- Quand une même méthode (par exemple `dessiner()`) est commune à tout une série de classes (par exemple, Rectangle, Carré, Ellipse, Cercle, etc.), cela signifie souvent qu'on peut introduire un niveau supérieur d'héritage plus abstrait (par exemple, *Forme*) et que c'est à ce niveau que la méthode doit être marquée.
- Les niveaux du dessous apportent chacun **leur propre** version spécifique de `dessiner()`.
- La méthode `dessiner()` est marquée `virtual` dans *Forme* afin de permettre le polymorphisme du tracé des formes.
-  Une méthode virtuelle est indiquée en *italique* dans le diagramme de classe.



- Marquer `dessiner()` dans `Forme` **impose** qu'on peut tracer n'importe quelle (descendant de) forme : c'est une spécification.
- Le hic, c'est qu'on ne sait absolument pas comment implanter la méthode `dessiner()` dans `Forme` (qui est bien trop générale pour qu'on sache **quoi/comment** tracer) !
- C'est une méthode virtuelle **pure** :
 - On la déclare dans `Forme` en rajoutant `=0` dans le `.hpp` :
`void dessiner()=0;`
 -  Son nom est ***en gras italique*** ;
 - On ne l'implante pas.
- Une classe dont (au moins) une méthode est virtuelle pure est dite **abstraite** : on ne peut pas l'instancier.
-  Une classe abstraite a son nom ***en italique***.
- On peut instancier les classes **concrètes** qui en dérivent !



- Up-casting : la conversion d'une classe vers sa parente (déjà vue) est toujours possible. Elle ne comporte pas de syntaxe particulière.
- Down-casting : la conversion **statique** d'une classe parente vers une classe fille est **impossible**...
- ... Mais la conversion **dynamique** est possible s'il s'agit d'un pointeur ou d'une référence et que l'instance désignée **est bien** du type souhaité et que la classe parente contient au moins une méthode virtuelle (polymorphisme).
- La syntaxe comporte des `<...>` qui trouveront leur explication à la fin du cours (template). Ne pas s'en préoccuper pour l'instant.

Exemples de down-casting

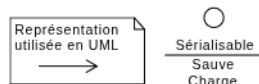
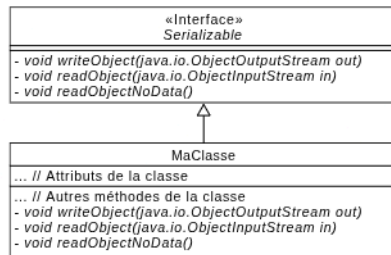
```
1  Forme      *f = new Cercle (...);           // Construction puis up-casting statique (légal)
2  Cercle     *c = (Cercle*) (f);              // Down-casting statique classique (illégal) : provoque une « error »
3  Cercle     *c = dynamic_cast<Cercle*>(f);     // Down-casting dynamique (légal)
4  Rectangle  *r = dynamic_cast<Rectangle*>(f);  // Down-casting (illégal car un Cercle n'est pas un Rectangle) : affecte nullptr à r !
5  Cercle&    rc = dynamic_cast<Cercle&>(*f);   // Down-casting (légal)
6  Rectangle& rr = dynamic_cast<Rectangle&>(*f); // Down-casting (illégal car un Cercle n'est pas un Rectangle) : lance un bad_cast !
```

Run-time Type Information (RTTI)

- À condition que la classe ait au moins une méthode virtuelle, il devient possible à un programme d'interroger **à l'exécution** le type de ses données.
- L'opérateur `typeid( )` appliqué à un variable permet d'obtenir en résultat une structure `std :: typeinfo`.
- Cette structure `typeinfo` propose quelques méthodes comme `std :: type_info :: name()` qui donne le nom de la classe dont cette variable est une instance... mais son résultat n'est pas imposé par la norme et il varie d'un compilateur à l'autre.
- Par contre, il est possible de comparer sans problème l'égalité entre les `type_info` de deux instances pour savoir si elles sont du même type.
- Ce type de mécanismes, plus développé dans les langages interprétés que compilés, se nomme **introspection** (phylo. : la capacité d'un individu à s'interroger sur lui-même).

Interface

- Une classe abstraite sert à imposer l'implantation de fonctions virtuelles dans une sous-classe pour qu'elle soit concrète.
- Une classe abstraite, appelée **interface** ne comportant **que des méthodes virtuelles** (et **aucun attribut**!) peut donc être utilisée pour imposer qu'une classe concrète soit capable d'effectuer un ensemble d'opérations attendues.
- Exemple : l'interface *serializable* impose que la classe dispose de fonctions pour sauvegarder et recharger ses attributs.
- En C++, on hérite d'une telle interface. En Java, on l'« implements ».
-  Une interface est parfois représentée avec un cercle au dessus de son nom et sans le cadre extérieur.

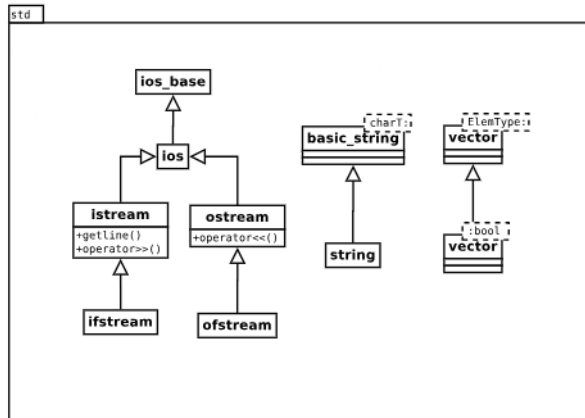


Espaces de noms (1/2)

- Un espace de nom (us : *name space*) correspond à la notion UML de *package* : un groupement thématique de classes, de types, de fonctions, de variables, etc.
- Tous les éléments (types, fonctions, instances, etc.) de la bibliothèque **standard** du C++ sont ainsi regroupés dans l'espace de nom « `std` » : le vrai nom de « `cout` » est par exemple « `std::cout` » !
- C'est pour éviter de devoir taper « `std::` » sans arrêt qu'on ajoute en général « `using namespace std;` » qui indique au compilateur « si tu ne connais pas un truc, regarde si tu le trouves dans `std::` ».
- Il est possible d'emboîter des espaces de noms ; c'est le cas des objets du *C++ Technical Report 1* qui sont définis dans « `std::tr1::` ».
- Dans le cas où il y a plusieurs espaces de noms actifs en même temps, je conseille de prendre son courage à deux mains et de marquer les noms complets *explicitement*.
- Sinon il est possible de panacher en utilisant une syntaxe moins systématique :
`using std::sin; // sin() sera prise dans std::`
`using ailleurs::cos; // alors que cos() sera prise dans ailleurs::`

Espaces de noms (2/2)

- Les classes que vous connaissez font partie du namespace « std ».
-  En UML, la notion d'espace de nom correspond à celle de **package** : une boîte avec étiquette qui contient des classes.



- Séparation complète entre :
 - la zone du programme où on détecte un problème et où on lance (avec le mot-clé `throw`) une erreur ;
 - la zone où on tente de faire une action qui peut provoquer l'erreur et où on va tenter d'y réagir si elle arrive (mots-clés `try {...} catch(type_erreur& e) {...}`).
- Attention : le débutant croit toujours que ces deux actions se passent au même endroit... mais ce n'est pratiquement jamais le cas !
- L'idée est qu'une erreur constitue une *exception* au déroulement normal du programme.

Lancement d'une exception

```
1 double inverse(double x)
2 {
3     if (x==0.0) throw "Division par zero !!!";
4     else      return 1/x;
5 }
```

- Le mot-clé `throw` doit être suivi d'une expression (ici une simple chaîne C) décrivant l'erreur rencontrée.
- L'instruction `throw` provoque la fin de la fonction (on ne fera donc jamais le `return` !).
- Lors du retour d'exception, la destruction des variables locales est *garantie* (ce ne serait absolument pas le cas si vous faisiez un `exit(1)` par exemple !).

- La norme décrit plusieurs classes d'exception héritant toutes de `std::exception`.
- Non seulement la valeur du message d'erreur mais également le **type** de l'exception lancée a son importance.
- Il est vivement conseillé de lancer des exceptions standards (**ou que vous en dérivez vous-même** si les existantes ne vous conviennent pas).

Lancement d'une exception

```
1 #include <stdexcept>
2
3 double inverse(double x)
4 {
5     if (x==0.0) throw domain_error("Division par zero !!!");
6     else      return 1/x;
7 }
```

- L'avantage des `std::exception` (par rapport à un type arbitraire quelconque) est que votre environnement de développement, débogueur, ... saura afficher l'erreur et vous permettra de décortiquer le problème (contrairement à un cas de « plantage ») !

- Les dérivées de la classe `std::exception` comportent un constructeur avec un paramètre chaîne de caractère permettant de préciser le message d'erreur.
- Elles ont également une méthode virtuelle `const char* what() const` qui permet de consulter ce message.
- Comme tout cela constitue un bel exemple de polymorphisme, le destructeur est lui aussi virtuel...
- Quelques classes :
 - `runtime_error` : une erreur ne pouvant pas être détectée plus tôt qu'à l'exécution ;
 - `domain_error` : un calcul est sorti de son domaine de définition (au sens mathématique du terme) ;
 - `range_error` : un paramètre est en dehors de la plage tolérable ;
 - `out_of_range` : l'erreur produite lorsqu'on sort des bornes d'un des conteneurs standards (vecteur, liste, etc.) ;
 - `bad_alloc` : l'erreur produite lorsqu'un « `new` » ne peut pas être satisfait ;
 - `bad_cast` : l'erreur produite lorsqu'un « `dynamic_cast<T&>` » ne peut pas aboutir ;
 - `underflow_error` / `overflow_error` : des erreurs de débordement arithmétiques ;
 - ...
- Plus de classes et de précisions sur www.cplusplus.com ou www.cppreference.com...

- Si on veut *tenter* une série d'opérations, dont on sait qu'elles pourraient rater, et qu'on sait ce qu'on devrait faire en cas de problème :
 - Le bloc `try {...}` indique les opérations à tenter
 - Le(s) bloc(s) `catch (...){...}` , qui doi(ven)t suivre immédiatement le bloc `try {...}` indique(nt) quoi faire en cas de problème.

Attraper une exception

```
1 int main()
2 {
3     try
4     {
5         double x; cin >> x;
6         cout << inverse(x) << endl;
7     }
8     catch(exception& e)
9     {
10        cerr << "Exception : " << e.what() << endl;
11    }
12    return 0;
13 }
```


- On peut même faire plusieurs tentatives :

Attraper une exception

```
1 int main()
2 {
3     double x;
4     bool recom;
5     do
6     {
7         recom = false;
8         cout << "Donnez un réel"; cin >> x;
9         try { cout << "1/" << x << "=" << inverse(x) << endl; }
10        catch(domain_error& de)
11        {
12            cerr << "Exception : " << de.what() << endl << "Recommence !" << endl;
13            recom = true;
14        }
15    }
16    while(recom);
17    return 0;
18 }
```

Au moment où on constate l'erreur :

- ❶ On crée une instance d'une classe dérivée de `std::exception` avec le plus d'information possible sur la cause de l'erreur ;
- ❷ On lance cette instance avec `throw` ;
- ❸ L'instance remonte la pile d'appels jusqu'à trouver un `catch` dont le type correspond, et ses instructions sont effectuées ;
- ❹ Si aucun `catch` compatible n'est rencontré, le `catch` « attrape tout » qui se trouve autour du `main()` sera utilisé... mais il arrête le programme (« Abnormal program termination ») !

Compléments :

- Possible de mettre plusieurs `catch` pour un même `try` : mettre les types les plus spécifiques **avant** les types les plus généraux sinon eux attraperont tout !
- Possible depuis un `catch` de relancer l'exception courante en mettant simplement « `throw;` ».
- Pour faire un `catch` « attrape tout », indiquez simplement « `...` » à la place du type du `catch`.

- Considérez les fonctions suivantes :

Fonctions similaires

```
1 void swap(int& a, int& b) { int temp(a); a=b; b=temp; }
2 void swap(double& a, double& b) { double temp(a); a=b; b=temp; }
3 void swap(string& a, string& b) { string temp(a); a=b; b=temp; }
```

- L'algorithme utilisé est le même ; seul le type des données (int, double, string) change !
- Le copier/coller est du temps perdu.
- **Pire !** C'est une source d'erreur !
- On doit pouvoir faire la même chose (et même plus) en étant plus malin...

- On écrit un patron de fonction `swap` qui généralise l'algorithme à n'importe quel type `T` :

Patron swap (swap.hpp)

```
1 template<typename T> void swap(T& a,T& b) { T temp(a); a=b; b=temp; }
```

- Un patron de fonction **n'est pas** une fonction, c'est un motif à répéter... pour fabriquer **autant** de fonctions `swap` que nécessaires !

Instanciation du patron swap (main.cpp)

```
1 #include "swap.hpp" // Inclusion de la déclaration du patron
2 int m=10,n=12; swap(m,n);
3 string s("pipo"),t("toto"); swap(s,t);
4 Triangle t1(...),t2(...); swap(t1,t2);
```

- L'exemple ci-dessus crée automatiquement **3 fonctions** (nommées instances du patron) :
`void swap<int>(int&, int&), void swap<string>(string&, string&),`
`void swap<Triangle>(Triangle&, Triangle&)` (le « `<...>` **fait partie** du nom !).

- Un patron **n'est pas** compilable. On ne peut que **le déclarer**. Sa place est donc dans un fichier « .hpp ».
- Vérification seulement au moment de la compilation de main.cpp que toutes les opérations nécessaires sont bien disponibles (ici : construction de copie et affectation).
- Dans l'exemple précédent, les paramètres des fonctions permettaient de choisir sans ambiguïté la bonne fonction ; si ce n'avait pas été le cas, il aurait fallu mettre le nom complet (avec le « <...> ») au moment de l'appel.
- On peut paramétrer un patron en fonction de types mais aussi de valeurs **constantes** (connues au moment de la compilation).

Pourquoi s'arrêter aux fonctions ?...

- Un patron de classe (**qui n'est pas une classe !**) permet de produire autant de classes que désiré et qui varient en fonction de types ou de constantes.
- `std::string` est en fait un `std::basic_string<char>` (et d'un deuxième argument) : on peut faire des chaînes d'autre chose que `char` ! Idem pour `istream`, `ostream`, etc.
- Même principe que pour les patrons de fonctions : un patron de classe **n'est pas compilable tel quel** alors sa place est aussi dans un fichier « `.hpp` ».
- La partie « `<...>` », qui fait partie du nom de chaque classe créée, **ne peut pas être sous-entendue** (en général) : ne l'oubliez pas sinon vous ne parlez pas de la classe mais de son patron !
- Un patron de classe contient des patrons de fonctions membres...
-  Une classe avec un rectangle pointillé dans le coin en haut à droite est en fait un patron de classe. Ce rectangle contient le(s) paramètre(s) du patron.

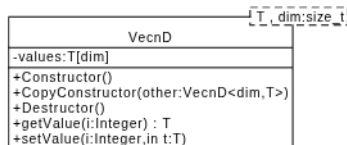
Un exemple : `VecnD<T,dim>` où

- `T` est le type des coordonnées du vecteur (`float` , `double` , `int` , `std::string` , etc.) et
- `dim` sa dimension (2D, 3D, etc.).

Patron `VecnD<...>` : instantiation

```

1 VecnD<int,10> t; // légal
2
3 const unsigned int max=100;
4 VecnD<char,max> tc; // légal car max est constant et connu à la compilation.
5
6 unsigned int max2=20;
7 VecnD<Triangle ,max2> tt; // illegal car max2 n'est pas constant.
```



- Deux façons de faire. Il est un peu plus facile de comprendre les erreurs de compilation lorsqu'on utilise celle de gauche.

Définitions internes des méthodes

```
1 // VecnD.hpp
2 template <typename T, size_t dim>
3 class VecnD {
4     T t[dim];
5 public:
6     const T& operator [] (size_t i) const
7     { if (i < dim) return t[i];
8       else throw std::range_error("..."); }
9 };
```

Déclarations externes des méthodes

```
1 // VecnD.hpp
2 template <typename T, size_t dim>
3 class VecnD {
4     T t[dim];
5 public:
6     const T& operator [] (size_t i) const;
7
8 };
9
10 template <typename T, size_t dim>
11 const T& VecnD<T, dim>::operator [] (size_t i) const
12 { if (i < dim) return t[i];
13   else throw std::range_error("..."); }
```

- Celle de droite a le mérite d'afficher de manière compacte les capacités du patron de classe (un peu comme sur un « .hpp » classique).

- On peut faire des spécialisations, c'est-à-dire donner des surcharges spécifiques à certaines combinaisons de valeurs de T ou n, par exemple avec un algorithme plus rapide dans un cas. Il suffit d'indiquer la combinaison dans le nom complet, par exemple :
`template<> int VecnD<double,3>::operator[](int) const {...}` est une spécialisation spécifique pour le type `double` et la dimension 3.
Attention : dans l'exemple précédent, la spécialisation étant complète, il n'y a alors plus de paramètre au template !
- On a parfois besoin de dire qu'une fonction n'est pas un patron, il suffit de rajouter « <> » à son nom pour indiquer qu'elle n'a aucun paramètre de patron.
- En pratique, commencez toujours par faire une classe ou une fonction « normale » et une fois qu'elle est bien au point, généralisez-la en patron. Écrire directement un patron est particulièrement difficile car les erreurs produites par le compilateur sont particulièrement difficiles à décrypter (longues et alambiquées) !

- La programmation générique est l'art d'écrire des algorithmes indépendamment de certains types : c'est l'équivalent C++ des Types Abstraits de Données étudiés en R1.01.
- Heureusement pour vous, beaucoup de choses ont déjà été écrites afin de vous éviter de réinventer la roue !
- En plus, une roue *parfaite*, c'est difficile à faire... alors autant profiter du travail de personnes plus compétentes que vous.

- Des patrons standardisés ISO garantis dans leur forme et qualité d'implantation (espace et temps) avec chaque compilateur : pour éviter de réinventer la roue !
- Une programmation très efficace... mais qui peut parfois devenir un peu technique.
- Comme toujours, n'inventez pas ! Cherchez l'information : livres, www.cplusplus.com, www.cppreference.com, etc.
- Petit tour d'horizon...



Des (patrons de) classes qui contiennent des éléments rangés dans un ordre précis (d'où le terme *séquence*), un peu comme dans un tableau.

- `std::vector<T>` : un tableau de T (compact en mémoire, accès en temps constant) capable de se ré-allouer au besoin.
- `std::stack<T>` et `std::queue<T>` : une pile et une file de T.
- `std::deque<T>` (double ended queue) : une séquence où l'ajout et le retrait d'un T est possible en tête ou queue de liste (donc une combinaison de `std::stack<T>` et `std::queue<T>`).
- `std::forward_list<T>` et `std::list<T>` : respectivement une liste simplement et doublement chaînée de T (insertion en temps constant mais recherche séquentielle).

Des (patrons de) classes qui contiennent des éléments rangés dans un ordre précis (d'où le terme *séquence*), un peu comme dans un tableau.

- `std::pair<T1,T2>` : une paire de deux éléments de types quelconques T1 et T2.
- `std::set<T>` : un ensemble d'éléments rangés dans un ordre particulier (implantation avec arbre binaire de recherche).
- `std::multiset<T>` : variante du précédent où il est possible de stocker plusieurs fois le même élément.
- `std::map<T1,T2>` : un tableau associatif (l'index, ou clé, n'est pas nécessairement un entier !), parfois appelé *dictionnaire* (car on peut s'en servir pour associer à une chaîne contenant un mot, une chaîne de définition). Implantation avec arbre binaire de recherche.
- `std::multimap<T1,T2>` : variante du précédent où il est possible de stocker plusieurs éléments pour une même clé.
- `std::unordered_set<T>`, `std::unordered_multiset<T>`, `std::unordered_map<T1,T2>`, `std::unordered_multimap<T1,T2>` : variantes C++ 2011 où l'ordre n'est plus assuré, ce qui permet un stockage éventuellement plus performant (listes avec tables de hachage).

- Chaque conteneur définit des *itérateurs* qui permettent de parcourir ses éléments.
- Un itérateur est la généralisation d'un pointeur : ça montre un élément... mais ça pourrait techniquement être autre chose qu'une adresse en mémoire.
- Mais en fait, cette subtilité n'est pas très importante : il suffit de s'en servir comme si c'était un pointeur !
- Certains conteneurs ont des variantes spécifiques : *forward* (avec lequel on ne peut que avancer) ou *reverse* (l'inverse), etc.
- Deux méthodes `begin()` et `end()` vous donnent accès au premier élément et à l'élément qui suit le dernier.
- Pour faire un parcours en arrière, il y a `rbegin()` et `rend()` qui vous donnent respectivement un `reverse_iterator` sur le dernier élément et sur l'élément qui précède le premier.
- En C++ 2011, des méthodes existent (`cbegin()`, `cend()`, `crbegin()` et `crend()`) aussi pour obtenir des `std::const_iterator` qui permettent la consultation mais pas la modification des éléments.

Itérateur : exemple

- Stocker des éléments dans un `std::vector<>`, les trier et les parcourir en sens inverse pour les afficher.
- Attention : l'exemple utilise des nouveautés de C++ 2011 !

Exemple

```
1 #include <string>
2 #include <vector>
3 #include <utility>           // Nécessaire pour sort()
4
5 using namespace std;
6
7 typedef vector<string> SequenceDeMots; // Un type pour stocker une séquence de mots.
8 SequenceDeMots mots;                // Définition d'une instance de ce type.
9 string mot;
10
11 while (cin >> mot)                // Tant qu'on arrive à lire un mot,
12     mots.push_back(mot);           // on l'ajoute à la suite de la séquence.
13
14 sort(mots.begin(), mots.end());    // Tri de la séquence indiquée par ses bornes (itérateurs)
15
16 typedef SequenceDeMots::const_reverse_iterator cri; // Alias pour simplifier. Remarquez la sous-classe.
17
18 for (cri i=mots.crbegin();         // Point de départ (itérateur reverse + constant)
19      i != mots.crend();             // Condition pour continuer.
20      ++i)                           // On « avance » d'un en un.
21     cout << *i << endl;           // On affiche l'élément désigné par i.
```

- Le C++ vous permet depuis la norme de 2011 d'utiliser deux sortes de « pointeurs intelligents » (*smart pointers*).
- Un *smart pointer*, exactement comme un pointeur classique, désigne un objet dans la mémoire grâce à son adresse : vous remplacez simplement « `int*` » par « `std::unique_ptr<int>` », par exemple.
- Ils servent à simplifier la vie quand on fait une allocation dynamique car...
- C'est encore vous qui allouez avec `new` mais...
- Vous confiez à votre *smart pointer* la responsabilité de libérer la mémoire (avec `delete`) quand l'objet n'est plus nécessaire.
- Vous n'avez alors plus à vous en soucier !

- Pointeur à responsabilité *unique* : une seule variable pointe sur l'objet dynamique. Elle en est **responsable**.

Utilisation basique

```
1 #include <memory> // Pensez à inclure ce fichier !
2
3 // Allocation d'une seule variable de type double
4 {
5     std :: unique_ptr<double> value(new double); // Allocation et affectation au
6                                                    // smart pointer.
7     *value = 3.1415; // Utilisation de la variable dynamique
8 } // Ici libération automatique, équivalente à « delete value; »
9
10 // Allocation d'un tableau de doubles
11 {
12     std :: unique_ptr<double[]> values(new double[1234]); // Allocation et affectation
13                                                            // au smart pointer.
14     values[0] = 3.1415; // Utilisation du tableau de variables dynamiques
15 } // Ici libération automatique, équivalente à « delete [] values; »
```

- La responsabilité se transmet par affectation : la source perd la responsabilité, la destination l'acquiert.
- Cette affectation n'est donc certainement pas une copie.

Transmission de responsabilité

```
1 {  
2     std :: unique_ptr<double> p1(new double); // p1 devient responsable.  
3     *p1 = 3.1415;  
4     std :: unique_ptr<double> p2 = p1; // La responsabilité passe de p1 à p2. p1 ne pointe plus  
5                                         // sur le double.  
6     // *p1 = 0.69; // Incorrect puisque ce n'est plus p1 qui montre la variable réelle.  
7     *p2 = 0.69; // Correct car p2 montre la variable réelle.  
8 } // Ici destruction de p2 puis p1.  
9 // La destruction de p2 entraîne la libération du réel.  
10 // La destruction de p1, qui ne désigne plus rien, n'a aucun effet supplémentaire.
```

- De manière plus commune, la transmission de responsabilité s'opère quand on reçoit le *smart pointer* comme valeur de retour d'une fonction.

- L'accès au pointeur « nu » se fait avec la méthode « get() ».
- Utile pour transmettre l'objet alloué dynamiquement à un code sans *smart pointer* (système d'exploitation, bibliothèque tierce, etc.)

Accès à la valeur d'adresse du pointeur

```
1 // On a une classe de donnée :  
2 class MyData { ... };  
3  
4 // On veut utiliser une fonction système qui réclame un « pointeur nu » :  
5 void attach_data(void *ptr_to_data) {};  
6  
7 std :: unique_ptr<MyData> data(new MyData (...)); // Construction des données  
8  
9 attach_data( data.get() ); // Appliquée au smart pointer, renvoie l'adresse « nue »
```

- La méthode est `const`, ce qui signifie qu'elle n'a pas le droit de changer l'instance de `unique_ptr<T>` mais le pointeur renvoyé comme résultat, lui, **ne l'est pas** !
- Attention à ne pas vous servir de cela pour contredire la responsabilité du `unique_ptr<T>` (par exemple en appelant vous-même `delete` dessus !)

- Le patron de fonction « std :: make_unique<T>(…) » permet d'allouer et construire directement l'instance en prenant les paramètres du constructeur de « T ».
- Cette construction « camoufle » l'appel à l'opérateur « new » et le déclenchement du constructeur de « T ». Les paramètres de cette fonctions sont repris de ceux du constructeur de « T ».
- Pour la classe « Ratio », on aurait :

Utilisation de « std :: make_unique<T>(…) »

```
1 // On conseille de remplacer :  
2 std :: unique_ptr<Ratio> pr(new Ratio(3,4)); // pr pointe sur le rationnel 3/4  
3 // par :  
4 std :: unique_ptr<Ratio> pr = std :: make_unique<Ratio>(3,4); // idem
```

- Cette construction a été ajoutée dans les normes de 2014 et 2023, dans un souci de cohérence avec l'utilisation de « std :: shared_ptr<T> »...

- Plusieurs pointeurs **partagent** la responsabilité d'un *unique* objet alloué dynamiquement.
- Le mécanisme, en plus de l'objet désiré, alloue un compteur :
 - chaque nouveau pointeur sur l'objet augmente le compteur de 1 ;
 - chaque pointeur détruit baisse le compteur de 1 ;
 - lorsque le dernier pointeur arrive en fin de vie, le compteur atteint 0 et l'objet pointé est automatiquement libéré.

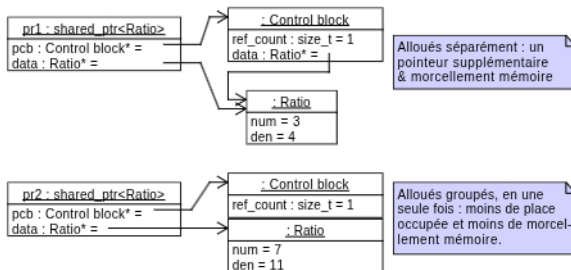
Utilisation basique

```
1 #include <memory> // Pensez à inclure ce fichier!
2
3 { // Allocation d'une seule variable de type double, adresse dans p1 :
4   std::shared_ptr<double> p1(new double(3.1415));
5   {
6     // Création de p2 pour désigner le même réel que p1.
7     std::shared_ptr<double> p2(p1);
8     *p1 = 0.69; // Utilisation de la variable dynamique
9     std::cout <<*p2; // Affiche 0.69.
10  } // Ici destruction de p2 et décrémentation du compteur.
11 } // Ici destruction de p1 puis libération du réel.
12
13 { // Allocation d'un tableau de réels.
14   std::shared_ptr<double[]> values(new double[1234]);
15   values[0] = 3.1415;
16 } // Ici libération automatique
```

- L'utilisation de « std :: make_shared<T>(...) » permet de « compacter » dans la même zone mémoire à la fois la variable dynamique et le bloc de contrôle qui contient le compteur de références. Cela économise du temps (un appel à new au lieu de deux).

Utilisation de « std :: make_shared<T>(...) »

```
1 // Éviter ceci :  
2 std :: shared_ptr<Ratio> pr1( new Ratio(3,4));  
3 // Au profit de ceci :  
4 std :: shared_ptr<Ratio> pr2 = std :: make_shared<Ratio>(7,11);
```



- Si deux objets alloués dynamiquement pointent chacun l'un sur l'autre avec des `std :: shared_ptr<T>`, même en supprimant le lien qui les rattache au reste, leurs compteurs de référence ne sont pas à 0 et donc ils ne seront pas détruits.
- Plus généralement, se méfier de tout cycle apparaissant dans la structure de données.
- Nécessité d'un pointeur sur l'objet mais sans en prendre **pleinement** la responsabilité.
- Utiliser `std :: weak_ptr<T>` pour garder un lien affaibli :
 - Il permet de copier un `std :: shared_ptr<T>` sans augmenter le compteur de référence ;
 - Pour utiliser l'objet, invoquer la méthode `std :: weak_ptr<T>::lock()` est nécessaire pour obtenir **temporairement** un nouveau `std :: shared_ptr<T>` (qui augmente le compteur de référence) si le `std :: weak_ptr<T>` est encore valable ou `nullptr` sinon. **Vérifiez avant d'accéder à l'objet !** Libérer/détruire le `std :: shared_ptr<T>` temporaire, dès que possible.

Utilisation

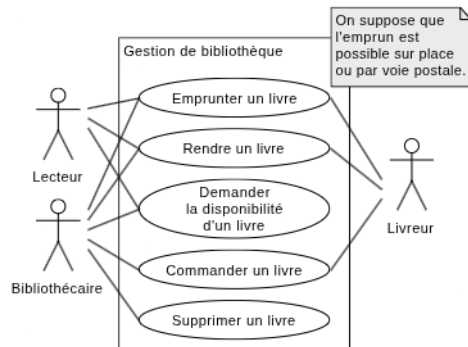
```
1 std :: shared_ptr<std :: string> sp = std :: make_shared<std :: string>("Bonjour");
2 std :: weak_ptr<std :: string> wp = sp; // wp désigne le même objet que sp mais sans le posséder.
3 // ... Ici l'objet pointé par sp peut ou non être détruit par ailleurs.
4 if (std :: shared_ptr<std :: string> sp2 = wp.lock()) // L'objet string existe-t-il encore ?
5     cout << "*sp2:" << *sp2 << " use_count=" << sp2.use_count() << endl;
6 else
7     cout << "wp has expired" << endl;
```

Ressource Allocation Is Initialization (RAII)

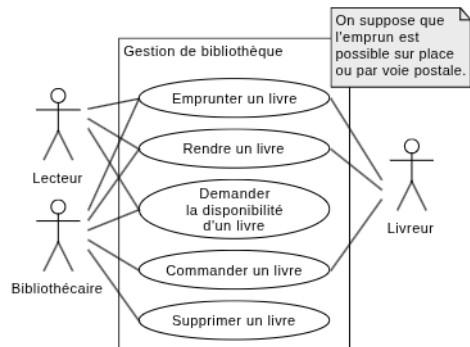
- L'allocation de la ressource se fait à la construction...
- ... et la restitution de la ressource est la destruction.
- Permet d'**assurer** la restitution des ressources... tant qu'on ne contrarie pas les mécanismes du fonctionnement interne de C++ (pas de `exit(int)`, `_exit(int)`, etc.).

- Parlons maintenant de **conception préliminaire** : recueillir et exprimer le **besoin du client**.
- Ce travail a lieu bien avant la conception technique (donc largement avant de produire le diagramme de classe) !
- La notation UML prévoit un diagramme qui rassemble de manière synthétique l'ensemble des scénarii d'utilisation qu'un système doit permettre : le **diagramme des cas d'utilisation / Use Case Diagram**.
- Le but est de lister l'ensemble des **exigences fonctionnelles** du système.
- L'ensemble des fonctionnalités à implanter est décrit dans un document contractuel : le **cahier des charges**.

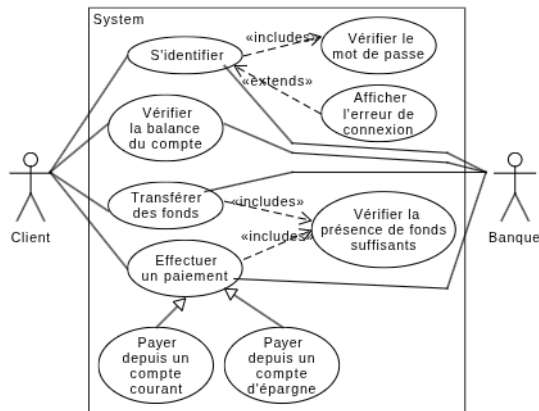
- Le diagramme liste les acteurs (symboles de « bonhomme bâton ») et montre les limites du système (le rectangle nommé).
- Un **acteur primaire**, ici le lecteur, est figuré à gauche du système. Les objectifs des cas d'utilisation lui sont essentiels.
- Un **acteur secondaire** est figuré à droite du système. Les objectifs des cas d'utilisation ne lui sont pas essentiels même si le système interagit avec lui.
- Un acteur n'est pas forcément humain : il peut être un système extérieur avec lequel le système modélisé interagit.
- Une même personne peut avoir plusieurs rôles (utilisateur et administrateur, par exemple) et donc être représentée par plusieurs acteurs.



- Le diagramme liste des cas d'utilisation, chacun sous forme d'un ovale.
- Les acteurs qui interviennent dans un cas d'utilisation lui sont liés par un trait. Le lien peut être orienté pour indiquer, de l'acteur ou du système, qui initie l'action.
- Un acteur secondaire ne déclenchant pas d'action du système, le lien est nécessairement orienté.
- Chaque scénario d'utilisation fixe un ensemble particulier de contraintes d'application pour un même cas d'utilisation. Un même cas d'utilisation intervient donc dans un ensemble de scénarii différents.



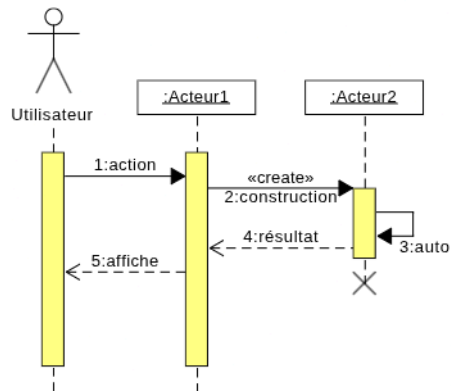
- Un cas d'utilisation inclus («include») est systématiquement effectué lorsque le cas qui l'inclut est effectué.
- Un cas d'utilisation qui étend («extends») un cas initial n'est effectué que lorsqu'un ensemble de conditions est réuni.
- **Attention au sens des flèches !**
- La généralisation permet de distinguer différentes variantes d'un cas d'utilisation ou d'acteurs.



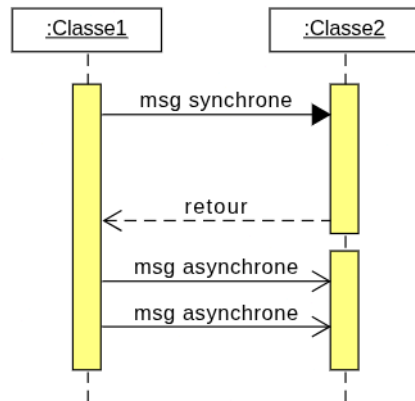
Cet exemple est tiré de <https://www.youtube.com/watch?v=zid-MVo7M-E>. La vidéo justifie les étapes de création du diagramme.

- Un diagramme de séquence UML décrit un comportement lié à une chronologie.
- Il illustre un scénario particulier, éventuellement avec quelques alternatives.
- Il représente une suite d'interactions qui s'enchaînent entre différents acteurs et qui sont nommées **messages**.
- En phase d'analyse du besoin, il permet par exemple de préciser le déroulement exact d'un cas d'utilisation.
- En phase de conception technique, les acteurs seront les classes (du diagramme de classe) et les messages échangés sont des appels aux fonctions membres. Les **messages de retour** sont alors les retours de fonction avec la valeur correspondante.

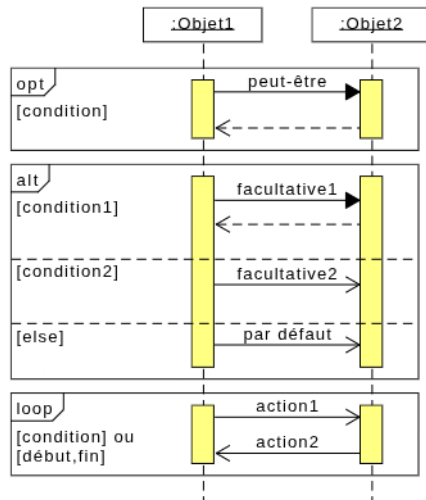
- Les acteurs du diagramme sont représentés en haut.
- Le temps s'écoule du haut vers le bas.
- La **ligne de vie** d'un acteur, en pointillé, représente son temps d'existence.
- Les rectangles verticaux sur la ligne de vie indiquent les **périodes d'activité** de l'acteur.
- Un stéréotype «create» indique, par exemple, la construction de cet acteur/classe.
- Les messages peuvent être numérotés.
- Un objet peut s'envoyer un message à lui-même.
- À l'instant indiqué par une croix, cet acteur disparaît.



- Un message synchrone (►) interrompt son émetteur jusqu'à ce qu'il reçoive une réponse. C'est le cas d'un appel de méthode d'une classe.
- Les messages de retour sont en pointillés.
- Un message asynchrone (➤) n'interrompt pas son émetteur (dans un programme *multithread* ou un système multi-processus).



- Une partie facultative (« opt ») de la séquence peut être indiquée dans un bloc avec la condition de sa survenue.
- Une partie alternative (« alt ») décrit plusieurs variantes avec les conditions de leurs déclenchements.
- Sur le même principe, on peut indiquer une partie qui se répète (« loop »).
- Un bloc parallèle (« par ») contient plusieurs sections qui se déroulent de manière concurrente.
- Un bloc « ref » peut faire référence à un autre diagramme détaillé par ailleurs pour simplifier le diagramme courant.
- De tels fragments peuvent être imbriqués (par exemple, une alternative dans une boucle).



UML comporte de nombreux autres diagrammes moins fréquemment utilisés :

- Diagramme d'objet (ou d'instances), vu en TD ;
- Diagramme de paquetage ;
- Diagramme de communication ;
- Diagramme de structure composite ;
- Diagramme d'états-transitions ou *state diagram*, vu en mathématiques R4.A.12 car similaire à la notation des automates ;
- Diagramme d'activité ;
- Diagramme de composants ;
- ...

Logiciels pour éditer les diagrammes UML :

- UMLet (www.umlet.com) : graphique + Markdown, rapide et performant (choix E. Remy)
- PlantUML (plantuml.com) : syntaxe texte spécifique (choix B. Desbenoit et R. Uribe)
- BOUML (www.bouml.fr) : gratuit, tout en graphique à la souris
- WinDesign : commercial, très puissant et plus général, installé en salle TP
- ...



- Les grands concepts découverts avec le C++ ont des équivalents dans les autres langages.
- Mais chaque langage a sa philosophie et ses particularités.
- Savoir programmer, c'est évidemment connaître les concepts et la syntaxe... mais c'est surtout avoir accumulé de *l'expérience*.
- Lisez les compléments de cours pour approfondir vos connaissances.

ALORS PROGRAMMEZ !

Des questions ?

- En théorie, vous savez maintenant concevoir et programmer *Orienté Objet* !
- En pratique, il va maintenant falloir vérifier que vous avez tout compris et retenu...

